Neural Networks 185 (2025) 107144

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

PFENet: Towards precise feature extraction from sparse point cloud for 3D object detection

Yaochen Li *, Qiao Li, Cong Gao, Shengjing Gao, Hao Wu, Rui Liu

School of Software Engineering, Xi'an Jiaotong University, Xi'an 710049, China

ARTICLE INFO

Keywords:
3D object detection
Sparse point cloud
Autonomous driving

ABSTRACT

Accurate 3D point cloud object detection is crucially important for autonomous driving vehicles. The sparsity of point clouds in 3D scenes, especially for smaller targets like pedestrians and bicycles that contain fewer points, makes detection particularly challenging. To solve this problem, we propose a single-stage voxel-based 3D object detection method, namely PFENet. Firstly, we design a robust voxel feature encoding network that incorporates a stacked triple attention mechanism to enhance the extraction of key features and suppress noise. Moreover, a 3D sparse convolution layer dynamically adjusts feature processing based on output location importance, improving small object recognition. Additionally, the attentional feature fusion module in the region proposal network merges low-level spatial features with high-level semantic features, and broadens the receptive field through atrous spatial pyramid pooling to capture multi-scale features. Finally, we develop multiple detection heads for more refined feature extraction and object classification, as well as more accurate bounding box regression. Experimental results on the KITTI dataset demonstrate the effectiveness of the proposed method.

1. Introduction

The recent 3D object detection methods use monocular images, RGB-D images or 3D point clouds for autonomous driving. The widely used 3D sensor is the LIDAR sensor, which can form sparse and disordered 3D point clouds to capture the 3D structure of the scene. How to extract discriminative features from irregular points is a challenge for 3D object detection.

3D object detection from point clouds can be divided into two categories for point cloud representations: grid-based methods and point-based methods. The grid-based converts irregular point clouds into 3D voxels (Sindagi, Zhou, & Tuzel, 2019; Zhou & Tuzel, 2018) or 2D bird-view images (Lang et al., 2019; Simony, Milzy, Amendey, & Gross, 2018; Yang, Luo, & Urtasun, 2018), which can be well processed by 3D or 2D CNN to learn features. Such methods are commonly used in 3D mapping tasks, such as those described in Jiang, Zhu, Lin, Li, and Guo (2020), to capture structural information of outdoor environments. Compared with point-based methods (Jiang et al., 2023; Qi, Litany, He, & Guibas, 2019; Shi, Wang, & Li, 2019; Yang, Sun, Liu, Shen, & Jia, 2019a; Yin, Zhou, & Krähenbühl, 2020) learning 3D representations from the original point clouds, it reduces computational complexity but sacrifices fine-grained localization accuracy. Thus, effective point cloud feature extraction is key to detection performance.

In 3D road traffic scenarios, detecting smaller objects, such as pedestrians and bicycles, is more challenging than detecting larger objects, such as cars. Small object detection in 3D scenes faces two main challenges: First, point cloud data is inherently sparse, especially for small objects, making effective feature extraction difficult. Second, small objects contain fewer points and appear less frequently in scenes, causing models to bias towards larger objects during training, which creates a class imbalance problem. To meet real-time requirements, we adopt a single-stage design, though this sacrifices the refined feature extraction capability of two-stage networks.

Thus, we propose PFENet, a single-stage voxel-based 3D object detector with enhanced point cloud processing. Our voxel feature encoder network(STAVFE) retains point cloud information within voxels while enhancing discriminative points and suppressing noise. A focal sparse convolutional layer (Chen, Li, Zhang, Sun, & Jia, 2022) is added to enhance dynamic sparsity learning and small object detection while maintaining model efficiency. The Attention Feature Fusion (AFF) module fuses spatial and semantic features, while atrous spatial pyramid pooling(ASPP) (Chen, Papandreou, Kokkinos, Murphy and Yuille, 2017) enables multi-scale prediction. Recent advancements in point set registration techniques (Zhu et al., 2019) further enhance multi-view feature fusion and improve detection accuracy. The network

* Corresponding author.
E-mail address: yaochenli@xjtu.edu.cn (Y. Li).

https://doi.org/10.1016/j.neunet.2025.107144
Received 30 September 2024; Received in revised form 20 December 2024; Accepted 10 January 2025
Available online 16 January 2025
0893-6080/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

combines Dense Head with ROI Head for improved feature extraction and classification. Experiments on KITTI test set demonstrate superior performance over existing single-stage voxel-based detectors for cars, bicycles, and pedestrians.

The key contributions of our work are as follows:

(1) We propose a robust voxel feature encoder network (STAVFE) which is capable of extracting voxel-wise features. Moreover, the stacked triple attention mechanism is employed to augment the crucial features within a voxel.

(2) By integrating Focal Sparse Convolution layers (FocalSpConv) into 3D sparse convolutions, the feature extraction ability of the sparse convolution layer is improved and richer 3D spatial feature maps are retained in the case of network deepening.

(3) We propose the attentional feature fusion (AFF) module, which adaptively fuses low-level spatial features and high-level semantic features and improves the network's multi-scale detection capability by using atrous spatial pyramid pooling(ASPP). Additionally, we introduce a Region of Interest (ROI) Head, employing multiple detection heads to refine the network's output.

2. Related work

3D Object Detection with Fusion-Based Methods. Some methods combine information from different sensors as input to the network. MV3D (Chen, Ma, Wan, Li and Xia, 2017) uses LIDAR point clouds and RGB images as input to encode sparse 3D point clouds into a compact multi-view representation. AVOD (Ku, Mozifian, Lee, Harakeh, & Waslander, 2018) integrates point cloud BEV maps and images, which can generate accurate region proposals for small classes of road scenes. F-PointNet (Cao, Chen, Zhang, & Wang, 2019) and F-ConvNet (Wang & Jia, 2019) include 2D detector and PointNet to evaluate the amodal 3D bounding boxs, but the final detection result relies heavily on the 2D detector, and the detection effect is extremely poor under bad lighting conditions. VFF (Li, Qi et al., 2022) maintains modal consistency by representing enhanced image features as rays in voxel fields and fusing them, thereby alleviating the constraint of point cloud sparsity in 3D object detection and better utilizing rich contextual clues from images. Bi-LRFusion (Wang et al., 2023) improves the performance of 3D dynamic object detection through a bidirectional radar and LiDAR data fusion framework, solving the main challenges of extremely sparse radar data and lack of high-level information when directly integrating radar data into a LiDAR centered detection network. SupFusion (Qin et al., 2023) uses "polar sampling" data augmentation technology to densify sparse objects and trains an auxiliary model to generate high-quality features as feature level supervision during the training process, effectively achieving the fusion of LiDAR and camera data, significantly improving detection performance. Unlike previous works, our network does not rely on RGB images as feature input, so it has better detection results for bad lighting conditions.

3D Object Detection with Point-based Methods. Point-based methods predict 3D targets directly from raw point cloud data using deep learning frameworks. PointRCNN (Shi et al., 2019) is used as a two-stage target detection algorithm to directly perform feature extraction and RPN operations based on the original point cloud data. 3DSSD (Yang, Sun, Liu, & Jia, 2020) proposes a lightweight and efficient single-stage 3d point cloud target detector that balances both accuracy and efficiency of detection. The framework removes the up-sampling layer and the refinement module to reduce the computation. A fusion adoption strategy in downsampling improves detection with fewer foreground points, which leads to better detection results on less representative points. Point-GNN (Shi & Rajkumar, 2020) proposes a graph neural network-based target detection method for LiDAR point clouds, which effectively encodes the point clouds in a fixed-radius nearest-neighbor map. STD (Yang, Sun, Liu, Shen, & Jia, 2019b) proposes a new two-stage 3D target detection framework called sparse to dense 3D target detection framework(STD). The first stage is a bottom-up proposal generation network that uses the original point cloud as input and generates accurate proposals by seeding each point with a new spherical anchor. The internal point features of the proposal features are then converted from sparse expressions to compact representations via PointsPool to generate proposal features, which allows efficient and effective prediction.

3D Object Detection with Grid-based Methods. In exploring grid-based methods for feature extraction from irregular LiDAR data, significant advancements have been made through various one-stage methods such as PIXOR (Yang et al., 2018), ComplexYOLO (Simony et al., 2018), and PointPillars (Lang et al., 2019), which convert point clouds into bird's-eye-view 2D data for processing with 2D CNNs. While these approaches, including the PointPillars method introduced by Lang et al. have significantly reduced computational demands by simplifying point cloud data into a pseudo two-dimensional image format, they also face the challenge of inevitable feature information loss during point cloud compression. Furthering the development in this domain, Li et al. proposed PillarNeXt (Li, Luo, & Yang, 2023), which refined network architectures and integrated 2D object detection modules into 3D detection, enhancing both accuracy and speed. Similarly, methods like VoxelNet (Zhou & Tuzel, 2018) and MVX-Net (Sindagi et al., 2019) applied 3D CNNs across divided 3D grids, leading to increased computational costs and diminished network performance, primarily because most grids are empty and contribute little to detection tasks. Addressing the inefficiencies in processing sparse 3D point clouds, SECOND (Yan, Mao, & Li, 2018) introduced sparse convolution and submanifold sparse convolution, tailored to the sparsity of 3D point clouds, thus achieving faster inference speeds. However, there remains room for better utilization of sparse convolution for feature extraction. In response, innovative frameworks such as Voxel R-CNN (Deng et al., 2021) by Deng et al. which leverages sparse voxel features, and OcTr (Zhou, Zhang, Chen, & Huang, 2023) by Zhou et al. utilizing an octree-based learning sparsity pattern, have markedly improved the efficiency and accuracy of 3D object detection. These frameworks have focused on optimizing the sparse convolutional network in 3D object detection and capturing rich global contextual information in a controlled computational complexity, respectively. These works aim to balance computational efficiency and detection performance while minimizing feature loss in LiDAR-based 3D detection.

3. PFENet for 3D object detection

3.1. Point cloud grouping

We obtain the voxel representation of the point clouds, following the processing the processes described in Zhou and Tuzel (2018). Point clouds are divided into 3D grids of the same size along x-axis, y-axis and z-axis, and then points in the voxel are randomly sampled. Assume that Point cloud in 3D space along the x-axis, y-axis, and z-axis ranges are W^*, H^*, D^*. At the same time, the voxel resolution used is v_W, v_H, v_D for three detection classes in same model. The corresponding the voxel grid size is $W = \frac{W^*}{v_W}, H = \frac{H^*}{v_H}, D = \frac{D^*}{v_D}$. We use

$$V = \left\{p_i = [x_i, y_i, z_i, r_i]^T \in R^4\right\}_{i=1\ldots t} \tag{1}$$

to indicate that a no-empty voxel contains $t \leq T_{\text{point}}$ LIDAR points, T_{point} means the maximum number of points in each voxel, p_i contains the XYZ coordinates x_i, y_i, z_i for the ith point and its reflection intensity r_i. A point is represented as a 10D vector to augment representation of the point p_i, referring to PointPillars (Lang et al., 2019). v_x, v_y, v_z represent the arithmetic mean of the XYZ coordinates of all points in a voxel, respectively. c_x, c_y, c_z represent the XYZ coordinates of the voxel

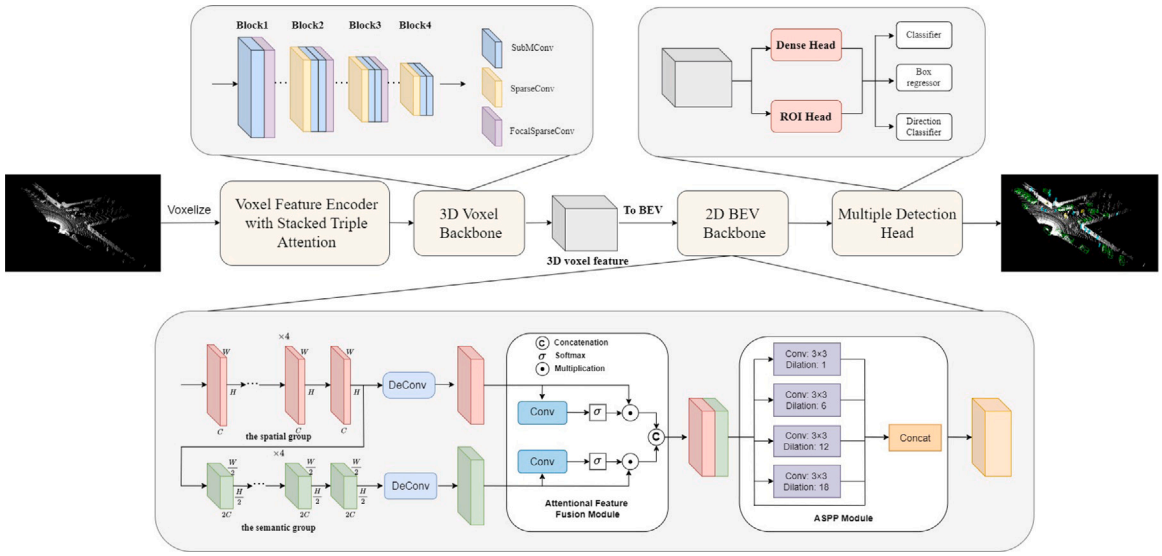

Fig. 1. The overall architecture of PFENet. The model takes point clouds as input and encodes them into voxelwise representations through voxel encoder network. The 3D voxel backbone network extracts the 3D spatial feature maps, and compresses the z-axis information into 2D features. The 2D BEV backbone network achieves robust extraction and fusion of semantic and spatial features, and expands the receptive field through atrous spatial pyramid pooling. Finally, multiple detection head generates the detection results.

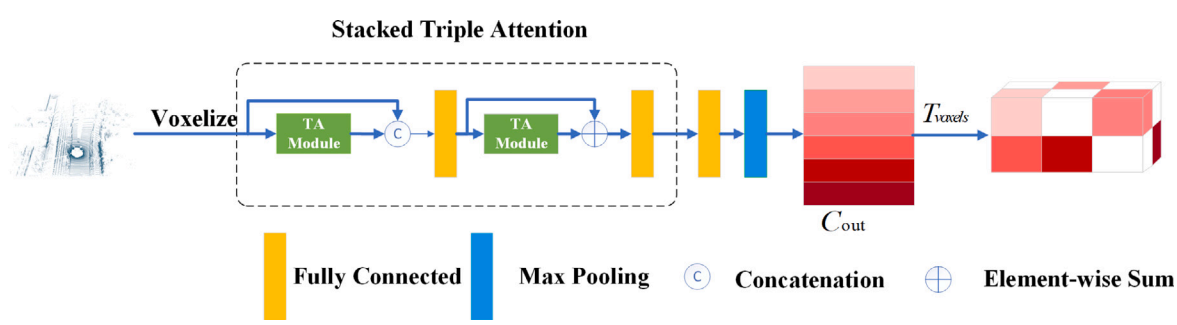

Fig. 2. The full pipeline of the voxel encoder network with stacked triple attention begins with a point cloud grouping operation. Next, the stacked triple attention mechanism is applied to refine the voxel representations, enhancing their discriminative power. Within each voxel, point features are aggregated using maximum pooling to generate a compact feature representation, denoted as $(T_{\text{voxels}}, C_{\text{out}})$. The feature vectors of different voxels are depicted in varying shades of red for clarity. These voxel features are then arranged based on their original spatial positions in the grid, resulting in a voxel feature map of size $C_{\text{out}} \times \text{H} \times \text{W} \times \text{D}$. The colored grids represent the compact feature vectors from the corresponding voxels, while the empty grid cells indicate regions with no data.

center, respectively. The input feature set of each voxel is

$$\text{V}_{\text{in}} = \left\{ \hat{p}_i = [x_i, y_i, z_i, r_i, x_i - v_x, y_i - v_y, z_i - v_z, x_i - c_x, y_i - c_y, z_i - c_z]^T \in \mathbb{R}^{10} \right\}_{i=1\ldots t} \tag{2}$$

Finally, the point clouds are encoded as pointwise features $(T_{\text{voxels}}, T_{\text{points}}, C_{\text{in}})$, where T_{voxels} is the maximum number of voxels in a mini-batch, and C_{in} is the size of the input point vector, so it is 10.

3.2. Network model

This section outlines our network structure, which is composed of four sub-networks: (1) a Voxel Feature Encoder with stacked triple attention; (2) a 3D Voxel Backbone; (3) a 2D BEV Backbone; and (4) Multiple Detection Heads. As shown in Fig. 1, we give the description of the overall architecture of PFENet.

3.2.1. Voxel feature encoder network with stacked triple attention

In this part, we mainly introduce our newly designed voxel feature encoder network(STAVFE), called Voxel Feature Encoder Network with stacked triple attention. As shown in Fig. 2, we give the full pipeline of Voxel Feature Encoding Network with stacked triple attention.

The point cloud grouping has been introduced in the previous section, so the feature of point clouds after voxelization has been encoded as pointwise features $(T_{\text{voxels}}, T_{\text{points}}, C_{\text{in}})$. Inspired by PointPillars (Lang et al., 2019), we design a new Voxel encoder network to extract the voxelwise features from the encoded pointwise features.

However, before the voxelwise features is extracted, a more discriminative and robust feature representation of each voxel can be learned for subsequent operations. By referring to the Liu et al. (2020), we have introduced stacked triple attention to the voxel encoder network, which utilizes multi-level feature attention. By considering the channelwise, pointwise and voxelwise attention jointly, the most important triple attention (TA) module enhances the crucial information of each voxel while suppresses the unstable cloud points. We stacked two TA modules to extract multi-layer features. The first TA module directly works on original features of point clouds, and the second operates on higher-dimensional features. For each TA module, we perform concatenate/sum operations on its output and input to fuse more feature information, and then obtain a high-dimensional feature representation through a fully connected layer. After passing the stacked triple attention module, we re-encode the voxelized point clouds into a 3D matrix $(T_{\text{voxels}}, T_{\text{points}}, C_{\text{ta}})$, where $C_{ta} = 64$ is the output dimension parameter of the TA module to extract the voxel feature configuration.

In terms of computational complexity, the stacked triple attention is designed to balance efficiency and feature enhancement. Each TA

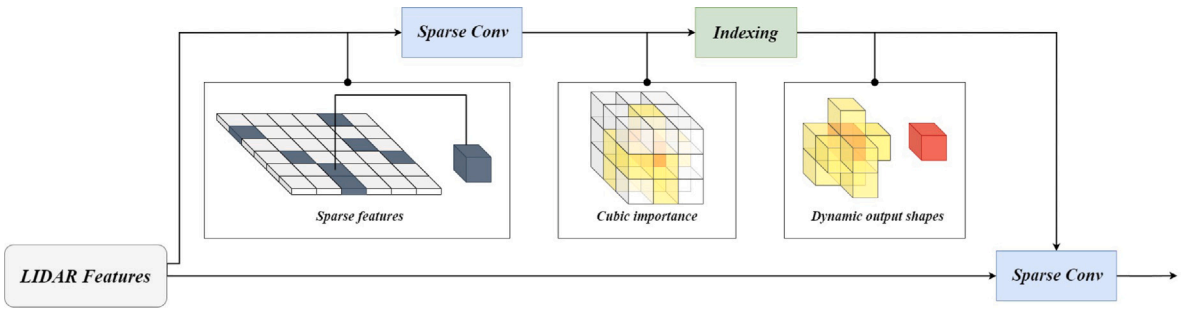

Fig. 3. Overview of focal sparse convolutional layer. The focus sparse convolutional layer predicts the cubic importance map of each voxel through additional convolution operations, and then dynamically adjusts its focus based on the map, selectively enhancing the processing ability of important features while suppressing unimportant background information. Finally, the features weighted by importance are further aggregated to generate the final sparse convolution output.

module introduces an additional cost proportional to the attention calculations across three dimensions: channelwise, pointwise, and voxelwise. Specifically, the channelwise attention computes a learnable weight vector for each feature channel, requiring a matrix–vector multiplication for each voxel. The pointwise attention involves applying a pointwise MLP (multi-layer perceptron) across all points in a voxel, which scales linearly with the number of points per voxel. Finally, the voxelwise attention aggregates information across all voxels in a local neighborhood, which introduces a dependency on the neighborhood size and voxel density. Despite these costs, the modularity of the TA mechanism ensures that the overall complexity remains manageable, and it can be parallelized efficiently on modern hardware such as GPUs.

Furthermore, compared to other attention mechanisms such as self-attention or multi-head attention, the stacked triple attention is unique in its hierarchical approach to feature refinement. While self-attention typically focuses on global feature relationships, the stacked triple attention explicitly disentangles the feature refinement process into three distinct levels: channel, point, and voxel. This separation allows for a more granular and focused enhancement of critical features while reducing the risk of overfitting or introducing noise from less stable points. Additionally, its lightweight design ensures that the stacked TA modules can be easily integrated into existing voxel feature extraction pipelines without significantly increasing the computational overhead.

After the stacked triple attention, similar to the network of PointNet, each point passes through the linear layer, the BatchNorm layer and the ReLU layer, and the output is a 3D matrix $(T_{\text{voxels}}, T_{\text{points}}, C_{\text{out}})$, C_{out} is set to 64. And we use channelwise max pooling to extract locally aggregated and symmetric features per voxel, in order to obtain voxelwise features $(T_{\text{voxels}}, C_{\text{out}})$.

3.2.2. 3D voxel backbone network with focal sparse convolution

The inherent sparsity and uneven distribution of 3D point cloud data, where a large number of points are concentrated in few areas while other regions are nearly devoid of points, pose significant challenges for point cloud processing. Although traditional sparse convolutions effectively reduce the wastage of computational resources by handling such sparse data, they often struggle to capture the detailed information within point clouds under uneven distribution conditions. Particularly in applications like autonomous driving, the precise detection of small objects, such as distant pedestrians or bicycles, is crucial. These small objects are typically represented by fewer points in the point cloud, further complicating the detection task. To address these issues, we introduce Focal Sparse Convolution(FocalSpConv) (Chen et al., 2022) layers following the nomal sparse convolutional layers, effectively tackling the problem. The workflow of Focal Sparse Convolution is illustrated in Fig. 3. It primarily consists of three steps: (i) cubic importance prediction, (ii) important input selection, and (iii) dynamic output shape generation.

Cubic Importance Prediction. This step aims to generate a cubic importance map for each output position of the convolution operation, matching the size of the convolution kernel. This importance map represents the contribution of input features to the output features in spatial terms. Specifically, the importance of input features surrounding each output position is learned through an additional convolution layer and can be represented by the formula:

$$I_{p0} = f_{importance}(X;\theta) \tag{3}$$

where $f_{importance}$ is the importance prediction function, X represents the input features, θ are the convolutional parameters, and I_{p0} is the predicted importance value.

Important Input Selection. Based on the predicted importance map, this step filters out those input features with higher importance, considered to be significantly contributory to the final task. This process can be achieved by setting an importance threshold τ, where only the features whose importance values exceed this threshold are selected as "important inputs". This reduces computational load and enhances efficiency since only important features are further processed:

$$P_{im} = \{p | I_{p0} \geq \tau\} \tag{4}$$

where P_{im} is the set of selected important input features.

Dynamic Output Shape Generation. Upon identifying the important input features, this step dynamically adjusts the output shape of the sparse convolution, allowing the model to focus on processing those features critical for the final task. This process is done based on the cubic importance map, determining which output positions should be retained. In this way, the model's output shape can dynamically adapt to different input features, optimizing the information flow:

$$K_{d_{im}} = \{k | I_{pk} \geq \tau, p + k \in P_{in}\} \tag{5}$$

where $K_{d_{im}}$ represents the dynamically chosen output shape based on importance, I_{pk} is the importance value at position p and its vicinity based on the offset k, and P_{in} represents the set of all positions of input features.

In comparison to other sparse convolution methods, Focal Sparse Convolution (FocalSpConv) introduces a novel capability for dynamic sparsity learning. Traditional sparse convolutions, such as Submanifold Sparse Convolution and standard Sparse Convolution, rely on fixed sparsity patterns defined by the voxelization process. This fixed nature limits their ability to adaptively prioritize critical regions within the point cloud, particularly under conditions of uneven spatial distribution. In contrast, FocalSpConv leverages the cubic importance prediction mechanism to dynamically identify and amplify the features that are most relevant for the task at hand. This adaptability enables the network to better capture fine-grained information, such as the sparse point representations of small or distant objects, while suppressing irrelevant background noise.

The introduction of FocalSpConv significantly bolsters the capacity for dynamic sparsity learning through three main mechanisms:

(1) Adaptive Importance Weighting: The cubic importance prediction map allows the model to assign varying levels of importance

to different spatial regions, enabling it to focus on areas with high feature density or task-relevant points. This ensures that computational resources are concentrated where they are needed most, improving feature extraction efficiency.

(2) Selective Feature Processing: By filtering out unimportant features based on the importance threshold τ, FocalSpConv reduces the overall computational burden while retaining the most critical input features. This not only enhances model efficiency but also minimizes the risk of overfitting to noisy or irrelevant data.

(3) Dynamic Output Adaptation: The ability to dynamically adjust the output shape of the convolution ensures that the network remains flexible in handling varying sparsity patterns across different input samples. This is particularly valuable in scenarios such as autonomous driving, where the density and distribution of point cloud data can vary significantly based on the environment.

Focal Sparse Convolution (FocalSpConv) directly addresses class imbalance between large and small objects by dynamically prioritizing features from sparse but critical regions. Through the cubic importance prediction mechanism, FocalSpConv assigns higher weights to regions with sparse point representations, such as small objects, ensuring their features are emphasized despite their limited points. This balances the feature extraction process, preventing large objects from dominating the learned representations.

Additionally, the selective feature processing and dynamic output shape generation further mitigate imbalance by focusing computational resources on task-relevant areas and ensuring that small objects' features are adequately propagated through the network. This results in improved detection accuracy for small objects (e.g., pedestrians, bicycles) without sacrificing performance on larger objects, making FocalSpConv particularly effective in applications like autonomous driving and robotics. The advantages of FocalSpConv are further highlighted in its application across various scenarios:

(1) Autonomous Driving: The ability to detect small and distant objects, such as pedestrians, cyclists, or traffic signs, is significantly enhanced by the dynamic sparsity learning of FocalSpConv. These objects are often represented by sparse and unevenly distributed points, making their detection a challenge for traditional methods.

(2) Robotics and Navigation: In robotics, precise object recognition in cluttered or uneven environments benefits from the selective feature processing capability of FocalSpConv, enabling robots to focus on task-relevant features while ignoring background clutter.

(3) Large-Scale Scene Understanding: In applications such as 3D mapping and urban planning, where point cloud data often covers large and complex environments, the dynamic output adaptation of FocalSpConv ensures efficient and scalable processing without sacrificing detail in critical areas. These enhancements translate directly into improved model performance, as demonstrated by the ability of FocalSpConv to achieve superior detection accuracy and efficiency compared to traditional sparse convolution methods. By dynamically learning to identify and prioritize important features, FocalSpConv enables the development of more robust and effective 3D point cloud processing models.

Together, these three steps form the core of FocalSpConv, optimizing the performance of sparse convolutional networks in processing 3D point cloud data, particularly for features with uneven spatial distribution. This provides a more precise and efficient processing method, thereby enhancing the model's detection performance for targets of various sizes.

Leveraging the introduced Focal Sparse Convolution, we construct a 3D voxel backbone network as illustrated in Fig. 1. Within this architecture, SubMConv stands for submanifold sparse convolution, SparseConv represents conventional sparse convolution, and FocalSparseConv denotes the focal sparse convolution we have introduced. The specific parameter configuration of the 3D voxel backbone network, composed of these three types of convolutional layers, is detailed in Table 1. In this table, it is important to note that in Block4, the SparseConv only adds one layer of zero padding to the boundaries of height and width, while no padding is added to the depth dimension.

Table 1
The network parameters of the 3D voxel backbone network. $\boldsymbol{k}, \boldsymbol{s}, \boldsymbol{p}$ represent kernel size, stride size, and padding size. C_{out} represents the number of output channels of the layer, and $\boldsymbol{N}$ represents the number of layers to be applied.

Block	Layer	k	s	p	Cout	N
Block1	SubMConv	3	1	1	16	1
Block1	FocalSparseConv	3	1	1	16	1
Block2	SparseConv	3	2	1	32	1
Block2	SubMConv	3	1	1	32	2
Block2	FocalSparseConv	3	2	1	32	1
Block3	SparseConv	3	2	1	64	1
Block3	SubMConv	3	1	1	64	2
Block3	FocalSparseConv	3	4	1	64	1
Block4	SparseConv	3	2	1	64	1
Block4	SubMConv	3	1	1	64	2

3.2.3. 2D BEV backbone network with attentional feature fusion module

After 3D voxel backbone network, we get the extracted 3D spatial feature map as the input of 2D BEV backbone network.

In order to detect objects, we have to achieve (1) regress the precise locations; (2) distinguish each regressed box as a positive/negative sample. Therefore, we consider low-level spatial features and high-level semantic features. A simple BEV feature extraction module obtain high-level semantic features with low-level spatial features declining in the final feature maps by simply stacking convolutional layers.

We introduce our 2D BEV backbone network, which includes two groups of convolution layers, a newly designed attentional feature fusion (AFF) module and an atrous spatial pyramid pooling(ASPP) module. These two convolutional groups are called the spatial groups and the semantic groups, and their respective outputs are called the spatial and semantic features. As shown in Fig. 1, we give an overview of the presented 2D BEV backbone network.

The Adaptive Feature Fusion (AFF) module is designed to dynamically combine spatial features and up-sampled semantic features, enhancing the robustness of feature extraction. After the 2D BEV backbone network with AFF module, the spatial and semantic features are shaped as $C \times W' \times H'$ and $C \times \frac{W'}{2} \times \frac{H'}{2}$, respectively. These features undergo processing through distinct pathways: the spatial features through spatial groups and the semantic features through semantic groups, each involving several 2D convolutional (Conv2D) operations. Subsequently, both feature sets are subject to a deblurring process via 2D transposed convolutions (ConvTranspose2d) to standardize the output shape. Following each Conv2D or ConvTranspose2d layer, the application of Batch Normalization and ReLU activation is mandatory.

Contrary to the baseline network that employs a straightforward concatenation of spatial and semantic features, our approach introduces a lightweight Attentional Feature Fusion (AFF) module in this sub-network. Initially, channel compression is applied to both feature sets, reducing their channel count to one, facilitating the generation of resultant vectors. These vectors are then normalized using a Softmax function to produce BEV attention maps, guaranteeing that all weight parameters fall within the [0, 1] range. Subsequently, these BEV attention maps serve as weighting factors for their corresponding features, upon which element-wise multiplication (denoted as $\odot$ in Fig. 1) is performed to yield updated spatial and semantic features. These enhanced features are then concatenated (symbolized as © in Fig. 1), culminating in the formation of the final feature set.

Additionally, to enhance the model's capability for multi-scale detection and further improve the detection performance of small objects, we process the fused features using Atrous Spatial Pyramid Pooling (ASPP). The structure of the ASPP module is shown in Fig. 4. Within this architecture, we first construct the ConvBlock submodule using 2D convolutional layers, batch normalization layers, and ReLU activation functions. Then, drawing on the concept of residual structures, we utilize ConvBlock to create another submodule named BasicBlock. In the ASPP module, feature maps initially undergo processing by the

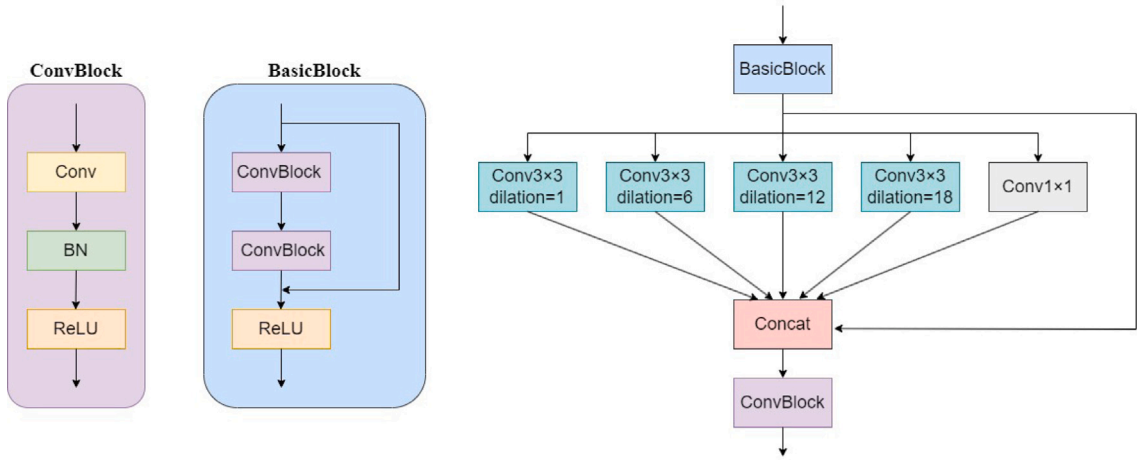

Fig. 4. Overview of the atrous spatial pyramid pooling (ASPP) module. Among them, ConvBlock and BasicBlock are sub modules within this module. The ASPP module contains four parallel branches of dilated convolutional layers, which use dilation rates of 1, 6, 12, and 18 to capture features at different scales. These four branches are fused with the output of a 1 × 1 convolutional layer branch in the depth direction (i.e. channel merging), then adjusted through the ConvBlock convolutional layer, and finally output advanced feature representations that consider multi-scale spatial context information.

BasicBlock module to elevate the level of feature abstraction, thereby aiding the subsequent dilated convolution branches in capturing more semantic information. This is followed by parallel branches of 3 × 3 convolutional layers with dilation rates of 1, 6, 12, and 18, alongside a branch of 1 × 1 convolutional layer. Finally, the outputs from all branches are concatenated and then processed through the ConvBlock module to produce the final output. By operating in parallel with convolutional kernels of varying dilation rates, the ASPP module effectively captures feature information across different scales, thus enhancing the model's robustness and generalization capability. Given that the feature map post-feature fusion contains both low-level detail and high-level semantic information, the ASPP module further enhances the semantic representation within these fused features. This allows the model to rely on richer and more accurate information for decision-making, enabling more precise object detection and thereby boosting the model's detection performance.

3.2.4. Multiple detection head

As depicted in Fig. 1, we elaborate on the architecture of the Multiple Detection Head. The primary advantage of this approach lies in its ability to merge the efficient feature extraction of the Dense Head with the precise object localization provided by the ROI Head. This combination allows the model to not only swiftly identify potential target areas within the voluminous point cloud but also to perform an in-depth analysis of these areas, leading to more accurate regression of the target's shape and position.

Refining the features extracted from point clouds is crucial for accurately classifying and localizing objects within complex scenes. This enhancement enables the model to effectively handle small or partially obscured objects, leading to a substantial improvement in detection accuracy. By integrating this approach, the model becomes adept at accommodating a wide range of object scales and densities, offering a robust solution for diverse 3D detection scenarios.

3.3. Muti-task loss function

For each category we set the same loss functions. The final multi-task loss function can be defined as follows:

$$L_{\text{total}} = \alpha L_{cls} + \beta \left(L_{reg-\theta} + L_{reg-other} \right) + \lambda L_{dir} \tag{6}$$

L_{cls} is the classification loss, which uses the focal loss function. $L_{reg-other}$ and $L_{reg-\theta}$ is the regression loss for localization and angle, they both use the smoothL1 function. And L_{dir} is the direction classification loss using the Softmax loss function. $\alpha = 1.0$, $\beta = 2.0$ and $\lambda = 0.2$ are the constant coefficients of our loss function.

4. Experiments

4.1. Setup and implementation details

Dataset. Our experiment uses the KITTI dataset, which contains 7481 training samples and 7518 test samples. Training samples are divided into the train split (3712 samples) and the val split (3769 samples). Three classes for evaluation, including cars, cyclists and pedestrians. The evaluation difficulty is divided into easy, moderate and hard according to bounding box height in the image plane, occlusion and truncation. We use the official KITTI evaluation metrics to evaluate the BEV and 3D test results with the average precision (AP).

Network Details. The point cloud range used along the x-axis, y-axis, and z-axis are $W^* = [0, 70.4], H^* = [-40, 40], D^* = [-3, 1]$ m. The size of the voxel is $v_W = 0.05, v_H = 0.05, v_D = 0.1$ m, resulting in the voxel grid size is $W = 1408, H = 1600, D = 40$. T_{point} sets to 5 as the maximum number of points in each voxel, and T_{voxel} sets to 16 000 as the maximum number of voxels in a mini-batch.

Training and Inference Details. The network we trained is on RTX 3090 GPU, the batch size is set to 4, and the number of training iteration is set to 80 epochs. The optimizer uses Adam, the initial value of the learning rate is 0.003, and then the learning rate will be updated to 0.9 times every 10 rounds, and the parameters of the optimizer are 0.95 and 0.85.

During training, we use data augmentation strategy, the points in a ground-truth are randomly rotated along the z-axis with a uniform distribution of $[-\pi/4, \pi/4]$ to obtain ground-truth orientation changes. Besides, we randomly flip point clouds in the 3D box along x-aixs. Also, according to random scaling of [0.95, 1.05], ground-truth boxes are randomly sampled and placed into the original samples to simulate a scene with multiple objects. We also used random "paste" some new ground-truth objects from other scenes to the current training scene for ground-truth sampling enhancements to simulate objects in various environments.

4.2. Comparison and evaluation

Comparison with state-of-the-art methods. As Table 2 demonstrates, our method significantly enhances detection performance on the KITTI test set through improved processing of point clouds and subnetworks. Our single-stage detection approach outperforms other single-stage methods, achieving a 2.07% increase in moderate difficulty car class

Table 2
Comparison with the state-of-the-art methods on the KITTI test set. The results are evaluated by the mean Average Precision with 40 recall positions.

Method	Type	Modality	Car-3D detection			Car-BEV			Cyclist-3D detection			Cyclist-BEV detection		
			Easy	Mod.	Hard	Easy	Mod.	Hard	Easy	Mod.	Hard	Easy	Mod.	Hard
F-PointNet (Cao et al., 2019)	2-stage	RGB + LIDAR	82.19	69.79	60.59	91.17	84.67	74.77	72.27	56.12	49.01	77.26	61.37	53.78
PointRCNN (Shi et al., 2019)		LIDAR only	86.96	75.64	70.70	92.13	87.39	82.72	74.96	58.82	52.53	82.56	67.24	60.28
3D IoU Loss (Zhou et al., 2019)		LIDAR only	86.16	76.50	71.39	91.36	86.22	81.20	/	/	/	/	/	/
Patches (Lehner et al., 2019)		LIDAR only	88.67	77.20	71.82	92.72	88.39	83.19	/	/	/	/	/	/
Fast Point R-CNN (Chen, Liu, Shen, & Jia, 2019)		LIDAR only	85.29	77.40	70.24	90.87	87.84	80.52	/	/	/	/	/	/
F-ConvNet (Wang & Jia, 2019)		LIDAR only	87.36	76.39	66.69	91.51	85.84	76.11	81.98	65.07	56.54	84.16	68.88	60.05
PV-RCNN (Shi et al., 2020)		LIDAR only	90.25	81.43	76.82	**94.98**	90.65	86.14	78.60	63.71	57.65	82.49	68.89	62.41
Voxel R-CNN (Deng et al., 2021)		LIDAR only	**90.90**	81.62	77.06	94.85	88.83	86.13	/	/	/	/	/	/
Focals Conv (Chen et al., 2022)		LIDAR only	90.55	82.28	77.59	92.67	89.00	86.33	/	/	/	/	/	/
OcTr (Zhou et al., 2023)		LIDAR only	90.88	**82.64**	77.77	93.08	89.56	86.74	/	/	/	/	/	/
ContFuse (Liang, Yang, Wang, & Urtasun, 2018)	1-stage	RGB + LIDAR	83.68	68.78	61.67	94.07	85.35	75.88	/	/	/	/	/	/
AVOD (Ku et al., 2018)		RGB + LiDAR	83.07	71.76	65.73	90.99	84.82	79.62	63.76	50.55	44.93	69.39	57.12	51.09
SECOND(baseline) (Yan et al., 2018)		LIDAR only	83.34	72.55	65.82	89.39	83.77	78.59	71.33	52.08	45.83	76.5	56.05	49.45
PointPillars (Lang et al., 2019)		LIDAR only	82.58	74.31	68.99	90.07	86.56	82.81	77.10	58.65	51.92	79.9	62.73	55.58
TANet (Liu et al., 2020)		LIDAR only	84.39	75.94	68.82	75.70	59.44	52.53	75.70	59.44	52.53	79.16	63.77	56.21
Associate-3Ddet (Du et al., 2020)		LIDAR only	85.99	77.40	70.53	91.40	88.09	82.96	/	/	/	/	/	/
UberATG-MMF (Liang, Yang, Chen, Hu, & Urtasun, 2019)		RGB + LIDAR	88.40	77.43	70.22	93.60	88.21	81.99	/	/	/	/	/	/
CenterNet3D (Yin et al., 2020)		LIDAR only	86.20	77.90	73.03	91.80	88.46	83.62	/	/	/	/	/	/
ACDet (Xu, Wang, Zhang, & Wan, 2022)		LIDAR only	88.47	78.85	73.86	92.87	89.21	85.80	83.80	66.61	59.99	87.76	71.48	64.69
CIA-SSD (Zheng, Tang, Chen, Jiang, & Fu, 2021)		LIDAR only	89.59	80.28	72.87	93.74	89.84	82.39	/	/	/	/	/	/
VoxSeT (He, Li, Li, & Zhang, 2022)		LIDAR only	88.53	82.06	77.46	92.70	89.07	86.29	/	/	/	/	/	/
Ours		LIDAR only	90.49	82.35	**79.77**	92.68	**90.79**	**88.49**	**86.13**	**67.87**	**63.41**	**90.75**	**72.35**	**67.79**
Improvement			−0.41	−0.29	+2.00	−2.30	+0.14	+1.75	+2.33	+1.26	+3.42	+2.99	+0.87	+3.10

Table 3
Results on the NuScenes validation data set.

Method	mAP↑	NDS↑	mATE↓	mASE↓	mAOE↓	mAVE↓	mAAE↓
BEVDet (Huang, Huang, Zhu, Ye, & Du, 2021)	42.2	48.2	0.529	0.236	0.396	0.979	0.152
BEVFormer (Li, Wang et al., 2022)	44.5	53.5	0.582	0.256	0.375	0.378	0.126
Far3D (Jiang et al., 2024)	51.0	59.4	0.551	0.258	0.372	0.238	0.195
CenterPoint (Yin, Zhou, & Krahenbuhl, 2021)	60.3	**67.3**	0.262	0.239	0.361	0.288	0.136
VoxelNeXt (Chen, Liu, Zhang, Qi, & Jia, 2023)	60.5	66.7	0.301	0.252	0.405	0.216	0.185
PFENet (Ours)	**60.7**	66.9	0.298	0.251	0.389	0.211	0.185

3D detection accuracy compared to the advanced CIA-SSD. Moreover, it performs comparably or even exceeds many two-stage methods in detection accuracy. And it is only 0.29% less accurate than the latest OcTr method for moderate difficulty car detection. Due to our method's single-stage nature, it also offers a speed advantage over two-stage approaches. Additionally, for the cyclist category, our method shows a notable increase in 3D detection APs on the KITTI test set, with improvements of 2.33%, 1.26%, and 3.42% respectively in easy, moderate, and hard levels, indicating its effectiveness in detecting small objects. In bird's-eye view detection for car and cyclist, our approach surpasses many previous advanced methods across all difficulty levels. Notably, we trained a single model for both car and cyclist detection, rather than separate models for each class as done in previous methods. These experiments validate the efficacy of our network on the KITTI test set.

Moreover, we conduct some experiments on the nuScenes dataset to evaluate the performance of the PFENet model. It can be seen from the Table 3 that PFENet performs quite well on the nuScenes dataset. Especially in terms of two key metrics, namely mAP (mean Average Precision) and NDS (nuScenes Detection Score), PFENet's performance is particularly outstanding. This demonstrates our model's high accuracy and robustness in handling complex scenes and diverse targets in the nuScenes dataset. These results further prove that PFENet not only performs excellently on the KITTI dataset but also can maintain a high level of performance on the more challenging nuScenes dataset, fully showcasing its good generalization ability and practical application potential.

Comparison of Voxel Feature Encoder. Here, we compare the effects of three voxel feature encoders, with the results presented in Table 4. In this table, PillarVFE represents a voxel feature encoder that we designed based on the concepts from the TANet network, which

Fig. 5. The qualitative results on the KITTI validation set. The predicted and ground-truth bounding boxes are displayed in red and blue respectively, and are projected back to the left camera image (line1 & line3 & line5) for visualization.

Table 4
Ablations on voxel feature encoder method in AP_{3D} (R11) on KITTI validation set.

Method	Car-3D detection			Cyclist-3D detection			Pedestrain-3D detection		
	Easy	Mod.	Hard	Easy	Mod.	Hard	Easy	Mod.	Hard
MeanVFE	87.12	77.67	76.47	79.11	63.68	60.26	54.17	50.82	46.12
PillarVFE	88.24	**78.37**	77.27	83.70	69.75	66.43	56.78	53.17	49.69
STAVFE	**88.28**	78.31	**77.38**	**85.06**	**70.48**	**67.40**	**59.12**	**56.34**	**52.16**

Table 5
3D detection performance: Average precision (AP) (in %) for 3D boxes in the KITTI validation set.

Method					Car - 3D detection			Cyclist - 3D detection			Pedestrian - 3D detection		
STAVFE	AFF module	ROI Head	FocalSp-Conv	ASPP module	Easy	Mod.	Hard	Easy	Mod.	Hard	Easy	Mod.	Hard
					87.12	77.67	76.47	80.18	65.80	61.30	56.30	51.75	48.81
√					88.02	78.15	77.09	79.52	65.29	61.65	55.28	51.55	47.15
√	√				88.73	78.63	77.47	80.91	65.38	62.05	56.41	52.50	47.93
√	√	√			**89.04**	78.75	77.67	85.41	65.54	62.23	55.31	49.32	45.63
√	√	√	√		88.81	78.90	77.91	**86.15**	69.87	66.14	**60.21**	54.45	49.80
√	√	√	√	√	88.61	**79.00**	**78.15**	85.06	**70.48**	**67.40**	59.12	**56.34**	**52.16**
Improve(compared with baseline)					+1.49	+1.33	+1.68	+4.88	+4.68	+6.10	+2.82	+4.59	+3.35

encodes point cloud features in a pillar format. As evidenced by the results, both STAVFE and PillarVFE encoding methods demonstrate significantly superior detection accuracy when compared to the MeanVFE encoding approach used in the SECOND network. Comparatively, STAVFE and PillarVFE exhibit similar precision in the detection of cars. However, STAVFE markedly outperforms PillarVFE in the detection of pedestrians and cyclists. This enhancement is attributed to the STAVFE voxel encoding network's integration of per-point features with a 10-dimensional vector, which enables more effective feature extraction from voxels, particularly for small objects with sparser point clouds.

Comparison of Detection Head. In the 3D object detection task, the detection head is a key component of the deep learning model architecture, which is responsible for identifying, classifying and localizing objects in 3D space from the features extracted from the model. Table 7 compares the effects of different detection heads on the experimental metrics. Dense Head adopts the original detection head in the baseline model. The method of Multi class head is used to design specialized detection heads for the three different classes in KITTI, which can learn feature representations that are more suitable for a specific class and improve the detection performance of objects in that class. The method using ROI Head allows for finer bounding box regression on candidate regions and improves localization accuracy by adding ROI Head to the model. In addition, ROI Head allows the model to perform further feature extraction on the candidate region, which helps to capture richer target features and thus improves the detection performance. In the KITTI dataset car is more numerous relative to Pedestrian and Cyclist, designing the detection head for each class may lead to insufficient performance enhancement of the model for a few classes, while the performance of the majority class may be degraded due to overfitting. Thus resulting in less than optimal experimental

Table 6
Bird's eye view detection performance: Average precision (AP) (in %) for BEV boxes in the KITTI validation set.

Method					Car - BEV detection			Cyclist - BEV detection			Pedestrian - BEV detection		
STAVFE	AFF module	ROI Head	FocalSp-Conv	ASPP module	Easy	Mod.	Hard	Easy	Mod.	Hard	Easy	Mod.	Hard
					89.18	86.93	85.50	83.47	69.51	66.40	62.60	56.17	53.78
√					89.81	87.70	86.46	81.87	69.48	64.83	61.56	55.69	52.70
√	√				89.91	87.72	86.21	82.73	68.19	64.29	62.06	56.89	54.10
√	√	√			**90.31**	87.55	85.75	87.07	70.73	66.46	61.05	54.39	49.98
√	√	√	√		90.11	87.97	86.59	87.94	75.25	70.73	**64.58**	57.95	55.14
√	√	√	√	√	90.12	**88.33**	**87.59**	**92.10**	**75.91**	**71.85**	64.06	**61.12**	**57.36**
Improve(compared with baseline)					+0.94	+1.40	+2.09	+8.63	+6.40	+5.45	+1.46	+4.95	+3.58

Table 7
Comparison of detection head methods in AP_{3D} (R11) on KITTI validation set.

Method	Car-3D detection			Cyclist-3D detection			Pedestrain-3D detection		
	Easy	Mod.	Hard	Easy	Mod.	Hard	Easy	Mod.	Hard
Dense head	88.73	78.63	77.47	80.90	65.38	62.05	56.41	52.50	47.93
Multi-Class head	88.10	78.13	76.43	76.89	61.46	57.62	**56.55**	50.77	46.72
Dense head + ROI head	**89.03**	**78.75**	**77.66**	**85.41**	**65.54**	**62.23**	55.31	49.31	45.63

metrics for the approach using Multi class head. By adding the ROI Head to the model, it can be seen that the metrics of both Car and Cyclist have improved to some extent, while the metrics of Pedestrian have decreased to some extent. It may be that the inclusion of ROI Head makes the model more focused on extracting features from the region of interest, which may be more favorable for relatively larger and more regular shaped targets such as Car and Cyclist, whereas pedestrians have more variations in pose and size, which may lead to a decrease in model performance. Focal convolution can solve the class imbalance problem, and then it is easier to focus on the detection of small targets. Therefore we tried to use ROI Head and Focal convolution together to solve the problem of reduced metrics on pedestrians.

Analysis of Visualization Results. In order to better evaluate the proposed model, we conducted tests on the KITTI validation set and have displayed our visualization results in Fig. 5. Through the results, we found that the proposed network achieves unexpected detection effects. (1) Severely occluded or severely truncated objects can achieve better detection results. (2) Some small objects at long distances can also be detected with high precision. These results demonstrate the effectiveness and advanced nature of the model we proposed.

4.3. Ablation study

Evaluation Metric. The models are all trained on the train split and evaluated on the val split on the KITTI data set. mAP with a rotated IoU threshold 0.7 for cars and 0.5 for cyclists and pedestrians is calculated with 11 recall positions. Tables 5 and 6 shows 3D and bird's eye view detection performance in the KITTI validation set.

Effects of Voxel Encoder Network with stacked triple attention. We verified the effectiveness of our voxel feature encoder network by comparing with the baseline. As shown in Tables 5 and 6, although there is a slight decline in accuracy for the Cyclist and Pedestrian categories, we have achieved significant breakthroughs in the most challenging Car category across all three difficulty levels. This strongly demonstrates the effectiveness of our proposed Stacked Triple Attention Voxel Feature Encoder(STAVFE). In the KITTI dataset, the number of point clouds for cars significantly exceeds that for pedestrians and cyclists. Consequently, we hypothesize that the decrease in detection metrics for the pedestrian and cyclist categories may be due to the dominance of larger objects during the feature extraction process when using attention mechanisms. This dominance likely makes the model more sensitive to features of these larger objects, while it struggles to capture adequate feature information from smaller targets (such as pedestrians and cyclists) that contain fewer point clouds. To effectively mitigate this issue, we have also made corresponding improvements in the 2D backbone network.

Effects of AFF Module. To mitigate the overshadowing of small object features by larger object features during the detection process, we incorporated the Attentional Feature Fusion (AFF) module into our 2D backbone network, the results of which are shown in Tables 5 and 6. It is evident that our designed AFF module enhances performance across all three categories at different difficulty levels, particularly for pedestrians and cyclists, where it effectively alleviates the previously noted decline in accuracy. The AFF module, by integrating spatial and semantic feature information, effectively highlights the features of smaller objects within the feature maps, enhancing the model's perception of small object features. This ensures that during the detection process, the features of small objects are not overshadowed by those of larger objects due to their lesser information content, while also balancing the feature disparities between large and small objects. Consequently, this improves the accuracy and robustness of small object detection.

Effects of ROI head. In two-stage detection models, candidate boxes are typically generated first, followed by further refinement via ROI pooling for box regression. In our model, we have eliminated this complex structure and retained only the ROI refinement functionality within the Head, employing a multiple detection head structure with both Dense Head and ROI Head to detect objects. This approach effectively enhances detection performance without significantly increasing model complexity. As evidenced in Table 5, the introduction of the ROI detection head has led to a 0.12% improvement in the medium level for cars compared to the previous stage, and an even more optimal performance in the easy level, with a 1.92% increase over the baseline model, thereby thoroughly demonstrating the efficacy of the multiple detection head architecture in our model.

Effects of Focal Sparse Convolutional. In the field of 3D object detection, class imbalance is a common issue. Compared to cars, pedestrian and cyclist categories often contain fewer point clouds due to the lower number of targets and smaller size, which significantly increases the difficulty of detection. To effectively mitigate this issue, we employed focal sparse convolution in our 3D backbone network. Results from Tables 3 and 4 illustrate that, while the improvement in detection accuracy for cars is modest, there is a significant enhancement in the detection accuracy for cyclists and pedestrians. At the moderate difficulty level, the 3D detection accuracy for bicycles increased by 4.33%, and the BEV (Bird's Eye View) detection accuracy improved by 4.52%. Similarly, the pedestrian 3D detection accuracy rose by 5.13%,

and the BEV detection accuracy by 3.56%. These results adequately demonstrate that focal sparse convolution can focus on smaller, more challenging targets, reducing the emphasis on easier-to-detect objects. This shift enables the model to concentrate more on identifying critical targets, thereby enhancing the quality and robustness of the detection outcomes.

Effects of ASPP Module. In an effort to further enhance the average precision across different difficulty levels for each category, we integrated the Atrous Spatial Pyramid Pooling (ASPP) module into the 2D backbone network. As indicated in Tables 5 and 6, there was a notable improvement in the average detection accuracy for each category. This also amply demonstrates the effectiveness of our designed ASPP module in enhancing the model's multi-scale detection capabilities.

Effects of Overall Model. Through this series of improvements, our proposed model has achieved significant enhancements in detection accuracy across all categories compared to the baseline model. Notably, in the medium difficulty level of 3D detection, the accuracy for the Car category has improved by 1.33%, while the Cyclist and Pedestrian categories have seen increases of 4.68% and 4.59%, respectively. This highlights the exceptional performance of our model in detecting small objects.

5. Conclusion

In this work, we propose PFENet, a single-stage voxel-based 3D object detector. By introducing a novel voxel feature encoder network (STAVFE), our model effectively extracts voxel-level features from point cloud data. The integration of stacked triple attention enhances voxel feature representation by applying channel-wise, point-wise, and voxel-wise attention mechanisms. Additionally, the incorporation of Focal Sparse Convolution layers on top of standard sparse convolutions strengthens the feature extraction capability and preserves more original information. The AFF module further improves the 2D backbone network by enabling it to capture robust features with rich spatial context. The use of an ROI Head refines the network's predictions, enhancing detection accuracy. These innovations enable PFENet to precisely extract sparse point cloud features, even for small objects such as pedestrians and bicycles, which typically contain fewer point cloud points. Experimental results on the KITTI dataset show that PFENet significantly outperforms many state-of-the-art methods in 3D object detection, demonstrating its effectiveness and potential for practical applications. In future work, we aim to incorporate 2D image information to enrich 3D point cloud features by augmenting them with 2D image-based attributes. This enhancement is expected to generate more comprehensive 3D point cloud representations, thereby improving object detection accuracy.

CRediT authorship contribution statement

Yaochen Li: Writing – review & editing, Validation, Methodology. **Qiao Li:** Writing – original draft, Validation, Methodology. **Cong Gao:** Validation, Methodology. **Shengjing Gao:** Validation, Methodology. **Hao Wu:** Visualization. **Rui Liu:** Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China under grant no. 62473307, and the Provincial Key Research and Development Program under grant no. 2023-YBGY-033.

Data availability

Data will be made available on request.

References

Cao, P., Chen, H., Zhang, Y., & Wang, G. (2019). Multi-view frustum pointnet for object detection in autonomous driving. In *2019 IEEE international conference on image processing* ICIP, (pp. 3896–3899). IEEE.

Chen, Y., Li, Y., Zhang, X., Sun, J., & Jia, J. (2022). Focal sparse convolutional networks for 3d object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 5428–5437).

Chen, Y., Liu, S., Shen, X., & Jia, J. (2019). Fast point r-cnn. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 9775–9784).

Chen, Y., Liu, J., Zhang, X., Qi, X., & Jia, J. (2023). Voxelnext: Fully sparse voxelnet for 3d object detection and tracking. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 21674–21683).

Chen, X., Ma, H., Wan, J., Li, B., & Xia, T. (2017). Multi-view 3d object detection network for autonomous driving. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 1907–1915).

Chen, L.-C., Papandreou, G., Kokkinos, I., Murphy, K., & Yuille, A. L. (2017). Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *40*(4), 834–848.

Deng, J., Shi, S., Li, P., Zhou, W., Zhang, Y., & Li, H. (2021). Voxel r-cnn: Towards high performance voxel-based 3d object detection. *vol. 35*, In *Proceedings of the AAAI conference on artificial intelligence* (pp. 1201–1209).

Du, L., Ye, X., Tan, X., Feng, J., Xu, Z., Ding, E., et al. (2020). Associate-3ddet: perceptual-to-conceptual association for 3d point cloud object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 13329–13338).

He, C., Li, R., Li, S., & Zhang, L. (2022). Voxel set transformer: A set-to-set approach to 3d object detection from point clouds. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 8417–8427).

Huang, J., Huang, G., Zhu, Z., Ye, Y., & Du, D. (2021). Bevdet: High-performance multi-camera 3d object detection in bird-eye-view. arXiv preprint arXiv:2112.11790.

Jiang, X., Li, S., Liu, Y., Wang, S., Jia, F., Wang, T., et al. (2024). Far3d: Expanding the horizon for surround-view 3d object detection. *vol. 38*, In *Proceedings of the AAAI conference on artificial intelligence* (pp. 2561–2569).

Jiang, Z., Lu, G., Liang, X., Zhu, J., Zhang, W., Chang, X., et al. (2023). 3D-togo: Towards text-guided cross-category 3d object generation. *vol. 37*, In *Proceedings of the AAAI conference on artificial intelligence* (pp. 1051–1059).

Jiang, Z., Zhu, J., Lin, Z., Li, Z., & Guo, R. (2020). 3D mapping of outdoor environments by scan matching and motion averaging. *Neurocomputing*, *372*, 17–32.

Ku, J., Mozifian, M., Lee, J., Harakeh, A., & Waslander, S. L. (2018). Joint 3d proposal generation and object detection from view aggregation. In *2018 IEEE/RSJ international conference on intelligent robots and systems* IROS, (pp. 1–8). IEEE.

Lang, A. H., Vora, S., Caesar, H., Zhou, L., Yang, J., & Beijbom, O. (2019). Pointpillars: Fast encoders for object detection from point clouds. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 12697–12705).

Lehner, J., Mitterecker, A., Adler, T., Hofmarcher, M., Nessler, B., & Hochreiter, S. (2019). Patch refinement–localized 3D object detection. arXiv preprint arXiv:1910.04093.

Li, J., Luo, C., & Yang, X. (2023). PillarNeXt: Rethinking network designs for 3D object detection in LiDAR point clouds. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 17567–17576).

Li, Y., Qi, X., Chen, Y., Wang, L., Li, Z., Sun, J., et al. (2022). Voxel field fusion for 3d object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 1120–1129).

Li, Z., Wang, W., Li, H., Xie, E., Sima, C., Lu, T., et al. (2022). Bevformer: Learning bird's-eye-view representation from multi-camera images via spatiotemporal transformers. In *European conference on computer vision* (pp. 1–18). Springer.

Liang, M., Yang, B., Chen, Y., Hu, R., & Urtasun, R. (2019). Multi-task multi-sensor fusion for 3d object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 7345–7353).

Liang, M., Yang, B., Wang, S., & Urtasun, R. (2018). Deep continuous fusion for multi-sensor 3d object detection. In *Proceedings of the European conference on computer vision* ECCV, (pp. 641–656).

Liu, Z., Zhao, X., Huang, T., Hu, R., Zhou, Y., & Bai, X. (2020). Tanet: Robust 3d object detection from point clouds with triple attention. *vol. 34*, In *Proceedings of the AAAI conference on artificial intelligence* (pp. 11677–11684).

Qi, C. R., Litany, O., He, K., & Guibas, L. J. (2019). Deep hough voting for 3d object detection in point clouds. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 9277–9286).

Qin, Y., Wang, C., Kang, Z., Ma, N., Li, Z., & Zhang, R. (2023). Supfusion: Supervised lidar-camera fusion for 3d object detection. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 22014–22024).

Shi, S., Guo, C., Jiang, L., Wang, Z., Shi, J., Wang, X., et al. (2020). Pv-rcnn: Point-voxel feature set abstraction for 3d object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 10529–10538).

Shi, W., & Rajkumar, R. (2020). Point-GNN: Graph neural network for 3D object detection in a point cloud. In *2020 IEEE/CVF conference on computer vision and pattern recognition* CVPR, (pp. 1708–1716). http://dx.doi.org/10.1109/CVPR42600.2020.00178.

Shi, S., Wang, X., & Li, H. (2019). Pointrcnn: 3d object proposal generation and detection from point cloud. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 770–779).

Simony, M., Milzy, S., Amendey, K., & Gross, H.-M. (2018). Complex-yolo: An euler-region-proposal for real-time 3d object detection on point clouds. In *Proceedings of the European conference on computer vision (ECCV) workshops*.

Sindagi, V. A., Zhou, Y., & Tuzel, O. (2019). Mvx-net: Multimodal voxelnet for 3d object detection. In *2019 international conference on robotics and automation* ICRA, (pp. 7276–7282). IEEE.

Wang, Y., Deng, J., Li, Y., Hu, J., Liu, C., Zhang, Y., et al. (2023). Bi-LRFusion: Bi-directional LiDAR-radar fusion for 3D dynamic object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 13394–13403).

Wang, Z., & Jia, K. (2019). Frustum convnet: Sliding frustums to aggregate local point-wise features for amodal 3d object detection. arXiv preprint arXiv:1903.01864.

Xu, J., Wang, G., Zhang, X., & Wan, G. (2022). ACDet: Attentive cross-view fusion for LiDAR-based 3D object detection. In *2022 international conference on 3D vision (3DV)* (pp. 74–83). IEEE.

Yan, Y., Mao, Y., & Li, B. (2018). Second: Sparsely embedded convolutional detection. *Sensors*, *18*(10), 3337.

Yang, B., Luo, W., & Urtasun, R. (2018). Pixor: Real-time 3d object detection from point clouds. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 7652–7660).

Yang, Z., Sun, Y., Liu, S., & Jia, J. (2020). 3DSSD: Point-based 3D single stage object detector. In *2020 IEEE/CVF conference on computer vision and pattern recognition* CVPR, (pp. 11037–11045). http://dx.doi.org/10.1109/CVPR42600.2020.01105.

Yang, Z., Sun, Y., Liu, S., Shen, X., & Jia, J. (2019a). Std: Sparse-to-dense 3d object detector for point cloud. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 1951–1960).

Yang, Z., Sun, Y., Liu, S., Shen, X., & Jia, J. (2019b). STD: Sparse-to-dense 3D object detector for point cloud. In *2019 IEEE/CVF international conference on computer vision* ICCV, (pp. 1951–1960). http://dx.doi.org/10.1109/ICCV.2019.00204.

Yin, T., Zhou, X., & Krähenbühl, P. (2020). Center-based 3d object detection and tracking. arXiv preprint arXiv:2006.11275.

Yin, T., Zhou, X., & Krahenbuhl, P. (2021). Center-based 3d object detection and tracking. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 11784–11793).

Zheng, W., Tang, W., Chen, S., Jiang, L., & Fu, C.-W. (2021). Cia-ssd: Confident iou-aware single-stage object detector from point cloud. *vol. 35*, In *Proceedings of the AAAI conference on artificial intelligence* (pp. 3555–3562).

Zhou, D., Fang, J., Song, X., Guan, C., Yin, J., Dai, Y., et al. (2019). Iou loss for 2d/3d object detection. In *2019 international conference on 3D vision (3DV)* (pp. 85–94). IEEE.

Zhou, Y., & Tuzel, O. (2018). Voxelnet: End-to-end learning for point cloud based 3d object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 4490–4499).

Zhou, C., Zhang, Y., Chen, J., & Huang, D. (2023). Octr: Octree-based transformer for 3d object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 5166–5175).

Zhu, J., Jiang, Z., Evangelidis, G. D., Zhang, C., Pang, S., & Li, Z. (2019). Efficient registration of multi-view point sets by K-means clustering. *Information Sciences*, *488*, 205–218.